



Universidad
Indoamérica

UNITAT 8 · FONAMENTS DE SISTEMES OPERATIUS

Xarxa: els sòcols

Universidad Tecnológica Indoamérica

Laboratori TD7

Dos modes de connexió

Amb connexió ·

SOCK_STREAM (TCP)

- La connexió s'estableix una vegada; després s'intercanvien dades
- Com un **telèfon**
- Un **flux de bytes** fiable, com una canonada: detecta errors, retransmet, manté l'ordre

Sense connexió ·

SOCK_DGRAM (UDP)

- Cada enviament és un missatge independent (**datagrama**)
- Com un **telegrama**
- Sense garanties: pèrdues, duplicats, desordre

Flux contra datagrames, mesurat

Sortida real

```
$ ./udp
TCP: 3 writes (one, two, three) → read #1 returns 11 bytes: "onetwothree"
UDP: 3 sendto (one, two, three) → recvfrom #1: 3 "one", recvfrom #2: 3 "two", recvfrom #3: 5
"three"
UDP: receive buffer 8192 bytes, 10000 sendto of 100 bytes succeeded, 8 received, 9992 lost
kernel counter Udp RcvbufErrors grew by 9992
```

- TCP no conserva els límits: **el protocol d'aplicació** marca on acaba cada missatge (una línia, una longitud...)
- UDP perd datagrames **sense avisar ningú**, fins i tot a la mateixa màquina

```
int socket(int domain, int type, int protocol);
```

domain	AF_UNIX : local, l'adreça és un camí · AF_INET : IPv4 · AF_INET6 : IPv6
type	SOCK_STREAM : amb connexió · SOCK_DGRAM : sense connexió · SOCK_RAW : paquets en brut (cal CAP_NET_RAW)
+ opcions	SOCK_NONBLOCK , SOCK_CLOEXEC (Linux, des de la 2.6.27): combinats amb a type
protocol	0 : tria el sistema · IPPROTO_TCP · IPPROTO_UDP

Correcció (diapositiva 182): «AF_UNIX: local (127.0.0.1)» barreja dues coses. 127.0.0.1 és AF_INET sobre la interfície de bucle local; AF_UNIX no fa servir IP.

Domini local: un fitxer de tipus sòcol

Sortida real

```
$ ./local fso.sock  
bind created fso.sock: a file of type socket (the 's' of ls -l), sun_path holds 108 bytes  
the server asks the kernel who connected: pid = mine, uid 1000 (my user)  
read 5 bytes; after close the file stays until unlink  
still there after close: yes
```

- `bind` crea una entrada al sistema de fitxers (la `s` de `ls -l`); queda fins a `unlink`
- El nucli diu al servidor **qui** s'ha connectat (`SO_PEERCRED`): així decideixen els permisos `systemd`, `D-Bus` o `/var/run/docker.sock`

Domini Internet: adreça + port

- Un port (16 bits) identifica el servei dins de la màquina
- Ports < 1024: només privilegiats
(`CAP_NET_BIND_SERVICE`); a Linux el límit és `net.ipv4.ip_unprivileged_port_start`
- Noms dels serveis: `/etc/services`

Sortida real

```
$ ./addr 7000
...
ip_unprivileged_port_start = 1024
sin_port = htons(80):      bind: Permission denied
...
```

SERVEI	PORT	AVUI
ftp	21/tcp	sense xifrar; es fa servir sftp o https
telnet	23/tcp	sense xifrar; substituït per ssh (22/tcp)
http	80/tcp	gairebé sempre redirigeix a https
https	443/tcp, 443/udp	TLS; HTTP/3 funciona sobre UDP (QUIC)

bind : adreça + port

C

```
int bind(int sock, const struct sockaddr *addr, socklen_t len)
{
    struct sockaddr { sa_family_t sa_family; char sa_data[14]; };
    struct sockaddr_un { /* AF_UNIX (= AF_LOCAL) */
        sa_family_t sun_family;
        char sun_path[108]; /* a path */
    };
    struct sockaddr_in { /* AF_INET */
        sa_family_t sin_family;
        in_port_t sin_port; /* network order */
        struct in_addr sin_addr; /* 4 bytes */
        unsigned char sin_zero[8];
    };
    /* AF_INET6: struct sockaddr_in6 (16-byte address) */
```

- Cada domini té la seva estructura; totes es passen com a `struct sockaddr *` amb la seva mida
- `struct sockaddr_storage` pot contenir qualsevol d'elles (per exemple, a `accept`)

Sortida real

```
$ ./addr 7000
...
sizeof: sockaddr 16, sockaddr_in 16, sockaddr_in6 28,
sockaddr_un 110, sockaddr_storage 128
```

Diapositiva 187: hi faltava htons

C

```
/* slide 187 */
s.sin_port = 7000;           /* wrong */
/* fixed (sock.c) */
s.sin_port = htons(7000);    /* network order */
s.sin_addr.s_addr = htonl(INADDR_ANY);
```

- La xarxa fa servir l'ordre *big-endian*; x86-64 i ARM64 són *little-endian* (ho vas veure al TD1 amb `od`)

0x1B58 llegit al revés és 0x581B = **22555**: el servidor escolta en un altre port i ningú no el troba.

Sortida real

```
$ ./addr 7000
port 7000 = 0x1b58; htons(7000) = 0x581b in memory order
sin_port = 7000:           the kernel bound port 22555
sin_port = htons(7000):    the kernel bound port 7000
...
```

De gethostbyname a getaddrinfo

C tds/td7/addr.c

```
const char *names[][2] = { { "localhost", "http" }, { "
for (int k = 0; k < 2; k++) {
    struct addrinfo hints = { .ai_family = AF_UNSPEC, .
    int e = getaddrinfo(names[k][0], names[k][1], &hints);
    exit_if(e != 0, gai_strerror(e));
    for (struct addrinfo *p = res; p != NULL; p = p->ai)
...
    freeaddrinfo(res);
}
```

Sortida real

```
$ ./addr 7000
...
getaddrinfo("localhost", "http"): IPv4 127.0.0.1 port 80
getaddrinfo("::1", "https"): IPv6 ::1 port 443
...
```

- **gethostbyname** (diapositiva 187): només IPv4, no és segur amb fils, fora de POSIX des del 2008
- **getaddrinfo** (RFC 3493): nom + servei → llista d'adreces IPv4 i IPv6, a punt per a **connect**

`int listen(int sock, int backlog);`

- Declara el servei al sistema: el sòcol passa a ser **passiu**, només espera connexions
- `backlog`: quantes connexions **ja establertes** poden esperar `accept` (Linux n'admet backlog + 1)
- Amb la cua plena, el SYN es descarta i el client el torna a enviar (1 s, 2 s, 4 s...)

Sortida real

```
$ ./backlog 4 8
listen(4), 8 clients connect
0.3 s, no accept yet:      accept queue 5 | clients ESTABLISHED 5, SYN_SENT 3
accept 5 (total 5), wait:  accept queue 3 | clients ESTABLISHED 8, SYN_SENT 0
accept 3 (total 8), wait: all 8 clients accepted
```

accept : treure una connexió de la cua

```
C
int accept(int sock, struct sockaddr *a, socklen_t *len);  /* a new descriptor */
```

- a i *len s'omplen amb l'adreça del client (o NULL si no cal)
- Es bloqueja fins que hi ha una connexió a la cua
- Retorna un **descriptor nou** per a aquest client; el d'escolta continua acceptant

```
C tds/td7/sock.c
```

```
/* 5. accept blocks until a client connects and returns
int client = accept(s, NULL, NULL);
exit_if(client == -1, "accept");
printf("connection accepted, closing\n");
```

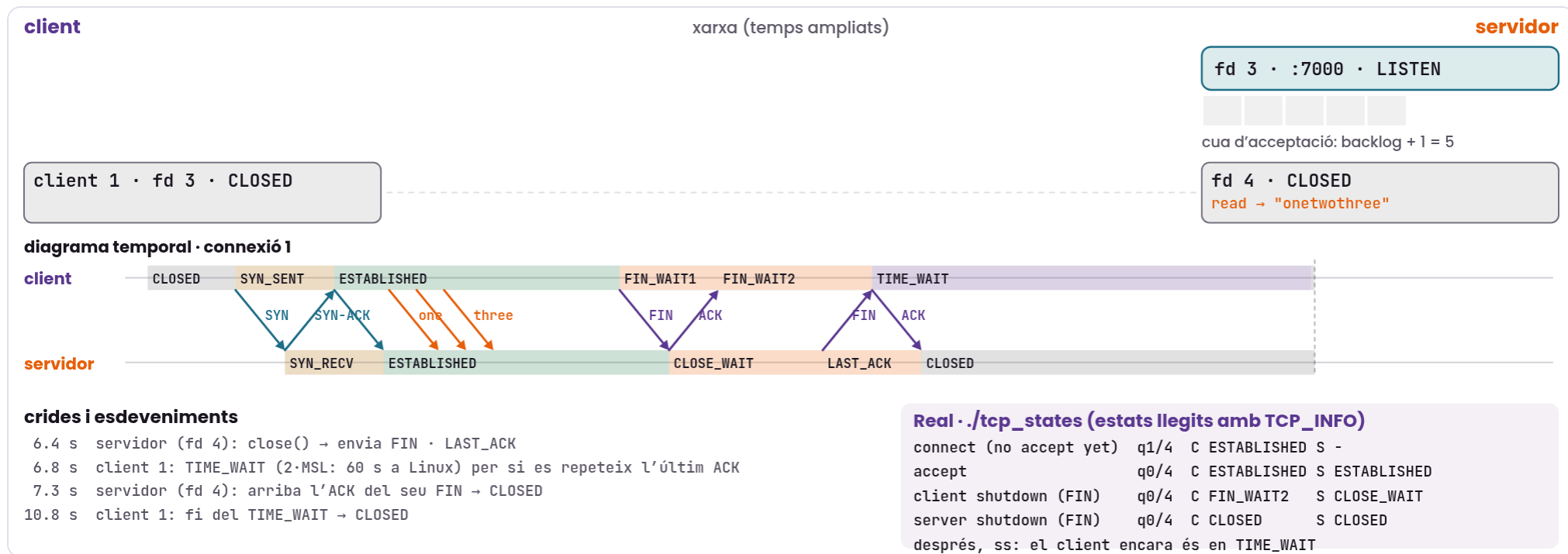
Correcció (diapositiva 189): retorna un descriptor ≥ 0 , no « > 0 » (el 0 és vàlid si s'ha tancat l'entrada estàndard), o -1 si hi ha error. `accept4(..., SOCK_CLOEXEC)` evita que el descriptor s'escolli en un `exec`.

connect : demanar la connexió

```
C  
  
int connect(int sock, const struct sockaddr *srv, socklen_t len);
```

- Demana la connexió a l'adreça del servidor: el nucli fa l'**encaixada de mans en tres passos** (SYN, SYN-ACK, ACK)
- Es bloqueja fins que acaba l'encaixada: **no espera** `accept`
- Errors típics: `ECONNREFUSED` (ningú no escolta en aquest port), `ETIMEDOUT`
- Amb `SOCK_NONBLOCK` retorna `EINPROGRESS` i l'encaixada continua sola (així és com `backlog.c` obre 8 connexions alhora)

Una connexió, segment a segment



Amb el descriptor que retorna `accept`

- Escriure: `write(fd, buf, n)` o `send(fd, buf, n, flags)`
- Llegir: `read(fd, buf, n)` o `recv(fd, buf, n, flags)`
- Tancar: `close(fd)` ; `shutdown(fd, SHUT_WR)`
només tanca la direcció d'escriptura (envia FIN)

El que saps de les unitats 2 i 5 s'aplica aquí:

- Lectures i escriptures **parcials**: bucle (`write_all`)
- `read` → 0: l'altre extrem ha tancat
- Escriure en un sòcol tancat → `SIGPIPE` ; s'evita amb `MSG_NOSIGNAL` o ignorant el senyal

Els estats, llegits del nucli amb TCP_INFO

Sortida real

```
$ ./tcp_states PORT; ss -tn state time-wait '( dport = :PORT )'
socket+bind+listen(4)      | listener LISTEN (queue 0/4) | client -          | server -
connect (no accept yet)    | listener LISTEN (queue 1/4) | client ESTABLISHED | server -
accept                     | listener LISTEN (queue 0/4) | client ESTABLISHED | server ESTABLISHED
write/read                  | server read 6 bytes
client shutdown (FIN)      | listener LISTEN (queue 0/4) | client FIN_WAIT2   | server CLOSE_WAIT
server read                 | server read returns 0: end of file
server shutdown (FIN)      | listener LISTEN (queue 0/4) | client CLOSED      | server CLOSED
                           | the client's end moved to a TIME_WAIT entry of the kernel (60 s, see ss)
Recv-Q Send-Q Local Address:Port Peer Address:Port
0      0      127.0.0.1:42772    127.0.0.1:PORT
```

Mode sense connexió: `sendto` i `recvfrom`

C

```
ssize_t sendto(int sock, const void *buf, size_t len, int flags,  
               const struct sockaddr *dest, socklen_t dlen);  
ssize_t recvfrom(int sock, void *buf, size_t len, int flags,  
                 struct sockaddr *src, socklen_t *slen);
```

- `sendto` = `connect` + `send` en una sola crida; `recvfrom` = `accept` + `recv`, i diu qui ha enviat
- Retornen els bytes enviats o rebuts (-1 si hi ha error); un `recvfrom` = **un datagrama sencer** (si la memòria intermèdia és curta, la resta es perd)
- També es pot fer `connect` d'un sòcol UDP: fixa el destí i permet `send` / `recv`

Qui crida què, en ordre

AMB CONNEXIÓ (TCP)	
SERVIDOR	CLIENT
socket · bind · listen	socket
accept (es bloqueja)	connect → encaixada
read ⇔ write	write ⇔ read
close	close

SENSE CONNEXIÓ (UDP)	
SERVIDOR	CLIENT
socket · bind	socket
recvfrom (es bloqueja)	sendto
sendto	recvfrom
close	close

Només el servidor lliga un port conegut; el client rep automàticament un port efímer.

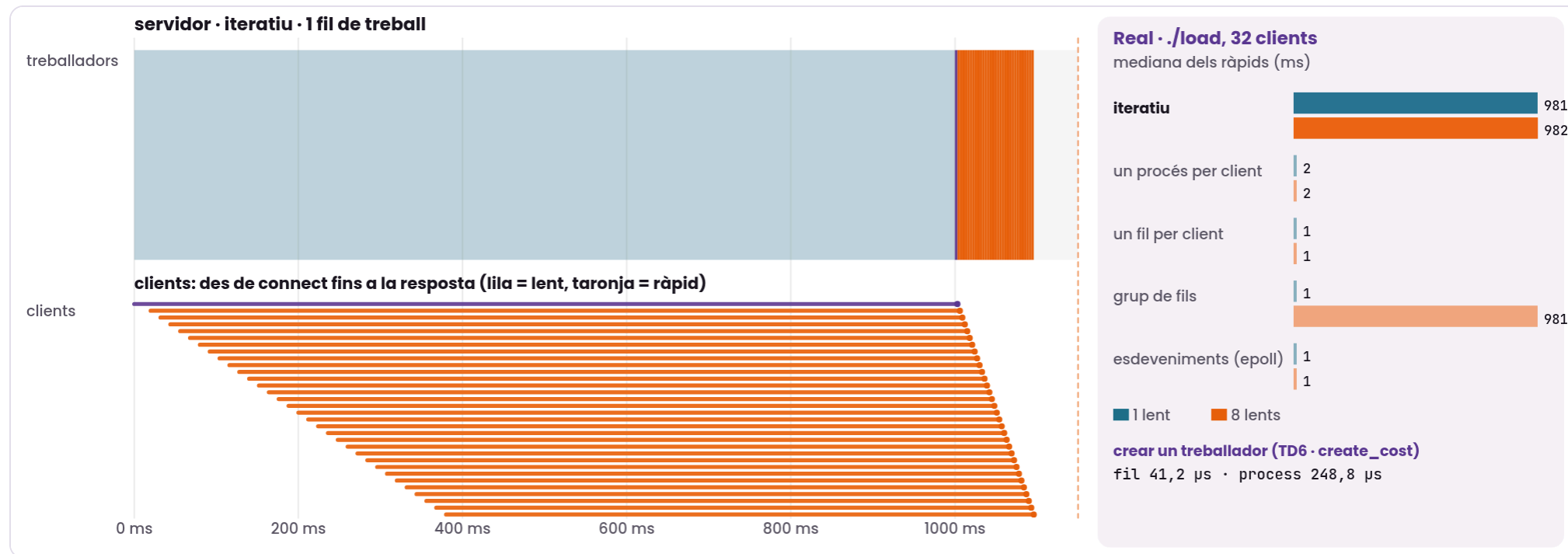
Models de servidor

- **Iteratiu**: un client cada vegada
- **Multiprocés o multifil**: el fil principal fa `accept`, crea un procés o un fil per al servei i torna a `accept`
- **Grup de fils**: el mateix, amb un nombre limitat de fils. Productor: el fil principal (connexions); consumidors: els fils de servei

Avui també

Bucle d'esdeveniments: un fil i `epoll` (Linux, 2002) serveixen milers de connexions; és la resposta al problema C10K (D. Kegel, 1999) i el model de nginx o Node.js. `io_uring` (Linux 5.1, 2019) porta la idea més enllà: cues compartides amb el nucli.

Qui espera qui?



32 clients; primer 1, després 8 de lents (1 s)

Sortida real

```
$ ./http_server -m iterative & ./load PORT 1 31 1000;  
./load PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 981 ms, max 982 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 982 ms, max 983 ms
```

Sortida real

```
$ ./http_server -m epoll & ./load PORT 1 31 1000; ./load  
PORT 8 24 1000  
32 replies 200 OK of 32  
slow 1: median 1001 ms, max 1001 ms  
fast 31: median 1 ms, max 2 ms  
32 replies 200 OK of 32  
slow 8: median 1001 ms, max 1001 ms  
fast 24: median 1 ms, max 2 ms
```

El servidor iteratiu fa esperar tothom per un sol client lent; `epoll`, amb un sol fil, no. El grup de 8 aguanta 1 client lent però no 8 (A8.2).

Conclusió

- Un sòcol és un descriptor: `read`, `write` i `close` com amb un fitxer o una canonada
- TCP: flux fiable, connexió feta pel nucli; UDP: datagrames sense garanties
- Adreces en ordre de xarxa (`htons`) i resoltes amb `getaddrinfo`
- El model de servidor decideix qui espera: processos, fils, grup o esdeveniments

TD7 resol't + repte IA

- sock , echo_server (iteratiu i amb fork)
- http_server amb cinc models
- Nou: tcp_states , backlog , load , udp , addr , local

Bash

```
make -C tds && tds/tests/run.sh td7
```



Universidad
Indoamérica

PER EMPORTAR-TE

Tres preguntes

1. Per què pot acabar `connect` abans que el servidor cridi `accept`?
2. Quin port obre `sin_port = 7000` al teu portàtil, i per què?
3. Quin model de servidor aguanta 8 clients lents amb un sol fil?